

Summer25(A)

1.a) Rushan wants to develop a server where everything that is being requested is handled by creating a child process of the original server process. The child process tends to complete and gets terminated, but after a while, Rushan notices that many inactive processes that have completed their task remain in the process table and are clogging up the system over time.

I. Identify what type of issue Rushan is facing with her system.

II. State how it can be solved.

I. Rushan is facing a **Zombie Process** issue.

II. It can be solved by ensuring the parent process **calls wait() or waitpid()** to read the exit status of child processes, or by **handling the SIGCHLD signal** to automatically clean up terminated child processes.

b) Observe the following code. **Calculate** how many processes and how many threads will be created. **[1*2
=2]**

```
main(){
    a = fork();
    b = fork();
    if (b > 0){
        pthread_create(...);
        fork();
    }
}
```

TOPIC NAME : Summer 2025(A)

DAY : _____

TIME : _____

DATE : _____

① b)

```
1 main() {
```

```
2     a = fork();
```

```
3     b = fork();
```

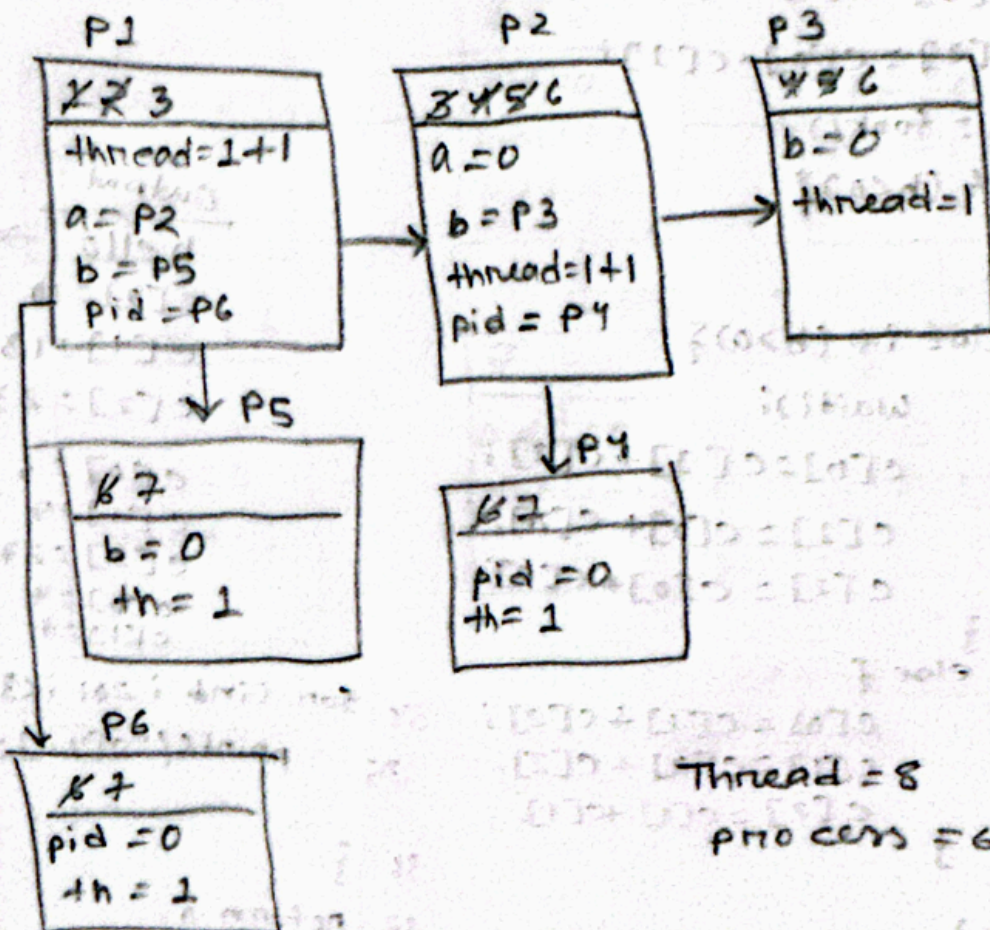
```
4     if (b > 0) {
```

```
5         pthread_create(...);
```

```
6         fork();
```

```
    }
```

```
}
```



c) A parent process forks, creating a child process. After the fork completes, the parent process creates a new shared memory segment and writes the number 111 into it.

I. Mention in yes or no if the child process now attempts to access that same shared memory segment, will it be able to see the number 111?

II. Justify the reason behind the previous answer.

I. No.

II. Because the child process **does not automatically inherit new shared memory segments** created by the parent after the fork(). Only shared memory segments that exist **before the fork** can be accessed by both processes; any segment created later is unknown to the child unless it explicitly attaches to it.

d) Find outputs of the given code:

[1*5
=5]

```
int main() {
    int counter = 0;
    int i;

    printf("Initial process starting. Counter: %d\n", counter);

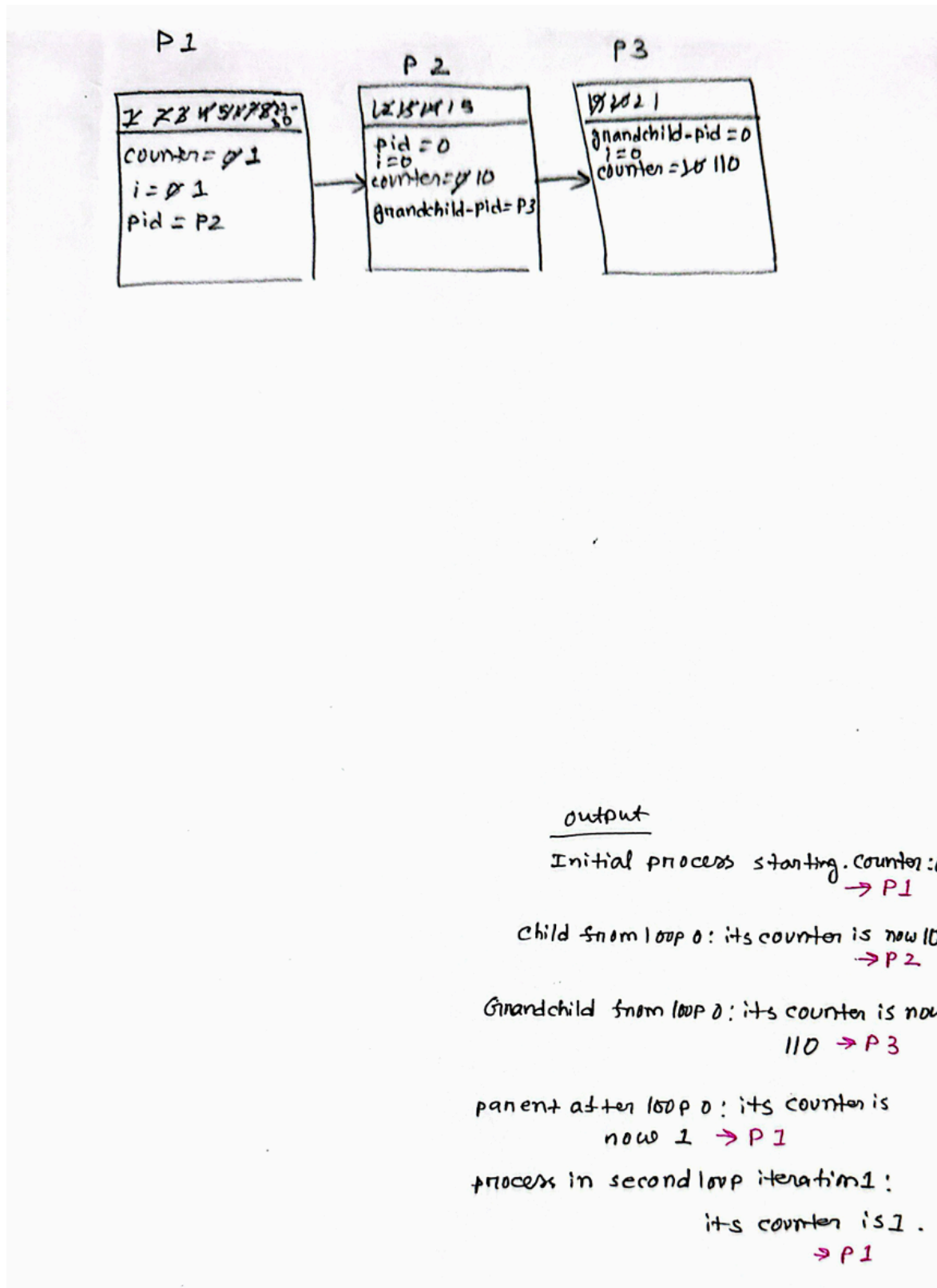
    for (i = 0; i < 2; i++) {
        if (i == 0) {
            pid_t pid = fork();

            if (pid < 0) {
                printf("Error forking.\n");
                return 1;
            } else if (pid == 0) {
                counter += 10;
                printf("Child from loop %d: its counter is now %d.\n", i, counter);
                pid_t grandchild_pid = fork();
                if (grandchild_pid < 0) {
                    printf("Error forking grandchild.\n");
                    return 1;
                } else if (grandchild_pid == 0) {
                    counter += 100;
                    printf("Grandchild from loop %d: its counter is now %d.\n", i, counter);
                } else {
                    wait(NULL);
                    counter += 1;
                }
            } else {
                wait(NULL);
                counter += 1;
                printf("Parent after loop %d: its counter is now %d.\n", i, counter);
            }
        } else {
            printf("Process in second loop iteration %d: its counter is %d.\n", i, counter);
            break;
        }
    }

    return 0;
}
```

d)

```
1 int main() {
2     int counter = 0;
3     int i;
4     printf("Initial process starting. Counter: %d\n", counter);
5     for (i = 0; i < 2; i++) {
6         if (i == 0) {
7             pid_t pid = fork();
8             if (pid < 0) {
9                 printf("Error\n");
10                return 1;
11            } else if (pid == 0) {
12                counter += 10;
13                printf("Child from loop %d: its counter is now %d.\n", i, counter);
14                pid_t grandchild_pid = fork();
15                if (grandchild_pid < 0) {
16                    printf("Error forking grandchild.\n");
17                    return 1;
18                } else if (grandchild_pid == 0) {
19                    counter += 100;
20                    printf("Grandchild from loop %d: its counter is\n", i, counter);
21                        now %d.\n", i, counter);
22                } else {
23                    wait(NULL);
24                    counter += 1;
25                }
26                break;
27            } else {
28                wait(NULL);
29                counter += 1;
30                printf("Parent after loop %d: its counter is now %d.\n",
31                        i, counter);
32            }
33        } else {
34            printf("Process in 2nd loop iteration %d: its counter is %d.\n", i, counter);
35            break;
36        }
37    }
38    return 0;
}
```



2.a) Write the answers in the question paper.

Suppose in a particular scenario FCFS CPU scheduler is being used.

- I. What significant drawback arises for short, I/O-bound processes when a long, CPU-intensive process is scheduled first using the algorithm?
- II. How does the issue lead to inefficient use of the CPU?

(I) Short, I/O-bound processes suffer long waiting times due to the Convoy Effect when a long CPU-bound process executes first.

(II) It causes inefficient CPU use because CPU and I/O devices remain idle alternately, reducing overall system utilization.

b) Write the answer in the answer script.

Shoumo is implementing his own OS. For his OS, he has decided to develop a **Multilevel Feedback Queue** as a scheduler. To design the scheduler, he decided to divide the ready queue into **three** levels. According to his preferences, in every level round-robin algorithm will be used, and some constraints should be maintained in each level, which are given below.

- **Q0:** time quantum = 4, priority = 19
- **Q1:** time quantum = 6, priority = 25
- **Q2:** time quantum = 8, priority = 29

Here, the lower priority value indicates the higher priority, and in case of an I/O request, the scheduler will promote a process by one level upon I/O operation completion. At a certain moment, Shoumo has collected information on burst times, arrival times, and I/O times of three processes.

Process	Burst Time	Arrival Time	I/O Time
P1	3	2	3 [After 1s of total CPU allocation]
P2	21	0	3 [After 13s of total CPU allocation]
P3	14	7	N/A

- I. **Draw** a Gantt chart using the multilevel feedback queue scheduling algorithm, showing the states of the ready queues of different levels. **[10]**
- II. **Calculate** the average waiting time and average turnaround time from the Gantt chart. **[1+1=2]**

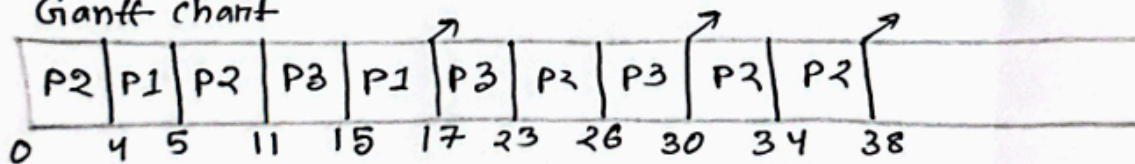
Summer 2025 (A)

(2) b) (i)

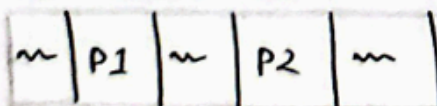
Process	Arrival time	Burst time	I/O time
P1	3	20 10	3 (after 1s of CPU)
P2	0	21 17 14 8 4 13 9 30	3 (after 13s of CPU)
P3	7	14 10 4 0	N/A

Q ₀ (RR:4)	Q ₁ (RR:6)	Q ₂ (RR:8)	I/O
0: P2	4: P2	11: P2	5: P1
2: P1	5: P2	15: P2	8: { }
5: { }	11: { }	17: P2	26: P2
7: P3	15: P3	23: P2, P3	29: { }
8: P3, P1	17: P3	26: P3	
15: P1	23: { }	30: { }	
17: { }	34: P2		
29: P2	38: { }		
34: { }			

Gantt chart



I/O Gantt chart



(ii)

Process	Turnaround time	Waiting time
P1	$17 - 2 = 15$	$15 - 3 - 3 = 9$
P2	$38 - 0 = 38$	$38 - 21 - 3 = 14$
P3	$30 - 7 = 23$	$23 - 14 = 9$
	Average = 25.33	Average = 10.67

3. a) Write the answers in the question paper.

[1*2

C02 Consider the following code for the test_and_set hardware tool.

=2]

```
boolean test_and_set(boolean *target){
    boolean rv=*target;
    *target=true;
    return *target;
}
```

```
do{
    while(test_and_set(&lock))
        ; //do nothing

    //critical section

    lock=false;

    //remainder section
}while(true);
```

- I. Identify the error in the code that will create issues in synchronization.
- II. Write the fix for the error.

I. Identification of the Error

The error lies in the **return value** of the `test_and_set` function.

In the provided code:

```
boolean test_and_set(boolean *target){
    boolean rv = *target;
    *target = true;
    return *target; // ERROR
}
```

Why this is an error:

The `test_and_set` instruction is supposed to return the **original (previous)** value of the target before it was updated to true. By returning `*target` (which has just been set to true), the function will **always return true**, regardless of whether the lock was available or not.

The consequence:

The process calling `while (test_and_set(&lock))` will enter an **infinite loop** at the entry section. Even if the lock is false (available), the function will return true, causing the process to "do nothing" and never enter the critical section.

II. The Fix for the Error

The function must return the local variable `rv`, which stores the value of the lock **before** it was modified.

Corrected Function

```
boolean test_and_set(boolean *target){
    boolean rv = *target; // Store the original value
    *target = true;       // Set target to true atomically
    return rv;            // Return the original value
}
```

How the fix works:

1. **If lock is false (Free):** rv becomes false. The function sets lock to true. It returns false. The while(false) loop terminates, and the process enters the **critical section**.
2. **If lock is true (Busy):** rv becomes true. The function keeps the lock as true. It returns true. The while(true) loop continues to spin, achieving proper **mutual exclusion**.

b) Write the answers in the question paper.

[1]

In a mobile banking application, one thread handles balance inquiries as follows:

```
    acquire(read_lock)
    show_balance(account.balance)
    release(read_lock)
```

Simultaneously, another thread processes cash-out transactions:

```
    acquire(write_lock)
    account.balance -= cashout_amount
    release(write_lock)
```

As a user, Alex is logged in on two separate devices. From one device, he initiates a withdrawal, and almost instantly, from the other device, he checks the current balance. Occasionally, the displayed balance appears outdated or incorrect.

Identify the probable reason for this behavior.

Thread 1: Balance Inquiry

```
acquire(read_lock)
```

```
show_balance(account.balance)
```

```
release(read_lock)
```

- This thread acquires a read_lock to safely read the balance.
- It then displays the balance to the user.

Thread 2: Cash-out Transaction

```
acquire(write_lock)
```

```
account.balance -= cashout_amount
```

```
release(write_lock)
```

- This thread acquires a `write_lock` to safely modify the balance.
- It subtracts the withdrawal amount and releases the lock.

Why the balance can appear outdated

1. Separate devices mean separate threads/processes

- Locks in a single process only synchronize threads **within that process**.
- If Alex is logged in on **two devices**, these threads may run on **different servers or processes**. Locks in one process **do not affect the other process**.

2. Read-after-write timing

- If the read thread executes **before** the write thread completes (or before it even starts), the balance shown will be **stale**, reflecting the old balance.

3. Lack of global synchronization

- The system seems to use **local locks**, which prevent inconsistencies **within one server thread**, but **not across distributed systems**.
- Mobile banking apps typically need **atomic transactions** or **distributed locks** to ensure all devices see the correct balance immediately.

c) Write the answers in the question paper.

[1*2

Consider the following code. According to this code, **find out** what values Thread 1 and Thread 2 will print.

=2]

```
int array[] = {12, 67, 88, 34, 91, 5, 48, 73, 55, 29};

void* find_and_print_values(void* arg) {

    //if it is Thread 1 then print the elements in the first half of the array whose values are > 50
    //else print the elements in the second half of the array whose values are > 50

}

int main() {
    pthread_t threads[2];
    int i;

    for (i = 0; i < 2; i++) {
        pthread_create(&threads[i], NULL, find_and_print_values, (void*)(i + 1));
    }

    for (i = 0; i < 2; i++) {
        pthread_join(threads[i], NULL);
    }

    return 0;
}
```

1. Analysis of the Data

The provided array is:

```
int array[] = {12, 67, 88, 34, 91, 5, 48, 73, 55, 29};
```

- **Total Elements:** 10
- **First Half (Indices 0–4):** {12, 67, 88, 34, 91}
- **Second Half (Indices 5–9):** {5, 48, 73, 55, 29}

2. Thread Execution Logic

The code creates two threads. The logic inside `find_and_print_values` depends on whether the thread is identified as Thread 1 or Thread 2 (passed via the `arg` parameter as `i + 1`).

Thread 1 (First Half)

- **Elements to check:** 12, 67, 88, 34, 91
- **Condition:** Value > 50
- **Filtered Values:** 67, 88, 91

Thread 2 (Second Half)

- **Elements to check:** 5, 48, 73, 55, 29
- **Condition:** Value > 50
- **Filtered Values:** 73, 55

3. Final Answers

According to the logic described in the comments:

- **Thread 1 will print:** 67, 88, 91
- **Thread 2 will print:** 73, 55